

The Calculus of Constructions I: The System

Lecture 16

Section 1

Part I: The Apex of the Cube

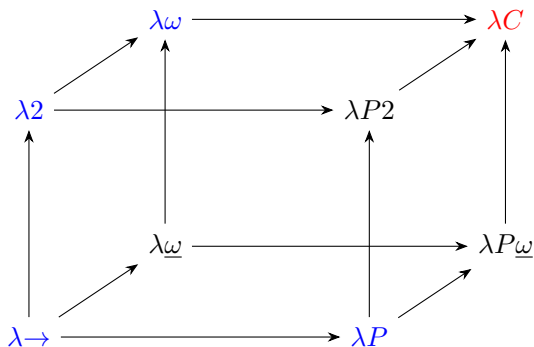
Recall: the λ -cube

We have explored four vertices of Barendregt's cube:

Vertex	Rules $\mathcal{R}$	Old name	Logic
$\lambda \rightarrow$	$\{(*, *)\}$	STLC	propositional
$\lambda 2$	$\{(*, *), (\Box, *)\}$	System F	2nd-order prop.
$\lambda \omega$	$\{(*, *), (\Box, *), (\Box, \Box)\}$	System F ω	higher-order prop.
λP	$\{(*, *), (*, \Box)\}$	—	1st-order pred.

Each vertex selects a subset of the four possible rules. Today we select **all of them**.

λC : the Calculus of Constructions



Blue: systems we studied. **Red:** today's destination. λC sits at the **apex**: every other vertex is a subsystem.

$$\mathcal{R}_{\lambda C} = \{(*, *), (\Box, *), (\Box, \Box), (*, \Box)\}$$

Rule	Dependency	Source system
$(*, *)$	terms on terms	$\lambda \rightarrow$ (STLC)
$(\Box, *)$	terms on types	$\lambda 2$ (System F)
$(\Box, \Box)$	types on types	$\lambda \underline{\omega}$
$(*, \Box)$	types on terms	λP

Only $\mathcal{R}$ changes: it is now as large as possible. The power of λC comes from the *interactions* between rules that were previously isolated in separate subsystems.

Historical context

λC was introduced by Thierry Coquand and G'erard Huet in 1986–88. Their motivation was practical: they wanted a single type theory that could serve as the **foundation of a proof assistant**.

We have seen the problem ourselves across the last few lectures:

- System F and $F\omega$ had polymorphism and could define data via Church encodings — but no dependent types, so one could not state or prove theorems about terms
- λP had dependent types and could express first-order logic — but no polymorphism, so every definition was stuck at a fixed type

Coquand and Huet's insight: combine *all* axes of the cube in a single system. The result was λC , and the proof assistant they built on it was **Coq** (1989).

Why one system matters

Huet and Coquand wanted a system where a user could:

- 1 **Define** data types (via Church encodings or, later, inductive definitions)
- 2 **Write** programs over them (via λ -abstractions)
- 3 **State** properties of those programs (via dependent types)
- 4 **Prove** those properties (via terms of the appropriate type)

all within a **single** formal language, checked by a **single** algorithm.

No subsystem of λC can do all four. λP does 1–4 for first-order properties but cannot define polymorphic data. $\lambda 2$ does 1–2 and defines data via Church encodings but cannot state dependent properties. λC is the *smallest* system in the cube that supports all four.

Section 2

Part II: Curry–Howard in λC

Recall: propositions as types, proofs as terms

The Curry–Howard correspondence has been our guiding thread. Let us recall how it has evolved:

In STLC, a type A is a proposition, and a term $t : A$ is a proof. The type $A \rightarrow B$ is implication; $\lambda x:A. t$ is an implication proof (assume A , derive B); and $f\ a$ is modus ponens. But the logic is very limited: there are no quantifiers, so we can only express propositional logic.

In System F, we added $\Pi\alpha:*. A$, which corresponds to universal quantification over *propositions*: *for all propositions α , A holds*. This gave second-order propositional logic.

In λP , we added $\Pi x:\alpha. P\ x$, which corresponds to universal quantification over *terms*: *for all x of type α , $P(x)$ holds*. This gave first-order predicate logic.

What λC adds

λC has *all* of these Π -types simultaneously. Since the rules do not interfere with each other (they just open new possibilities for well-formed Π -types), we can **nest** them freely.

For example, we can write:

$$\Pi\alpha:*. \Pi x:\alpha. \Pi P:(\alpha \rightarrow *). P\ x \rightarrow P\ x$$

This type mixes three forms of quantification:

- $\Pi\alpha:*$ quantifies over a type — requires $(\square, *)$
- $\Pi x:\alpha$ quantifies over a term — requires $(*, *)$
- $\Pi P:(\alpha \rightarrow *)$ quantifies over a predicate — requires $(\square, *)$ for the binder, and $(*, \square)$ to form the kind $\alpha \rightarrow *$

No subsystem has all the rules needed. $\lambda 2$ lacks $(*, \square)$, so the kind $\alpha \rightarrow *$ cannot be formed. λP lacks $(\square, *)$, so neither $\Pi\alpha:*$ nor $\Pi P:(\alpha \rightarrow *)$ can be formed.

The order of quantification

In first-order logic, we quantify over *individuals*: *for all x in α , $P(x)$* .

In second-order logic, we also quantify over *predicates*: *for all predicates P on α , ...*

There is no reason to stop. A predicate on predicates has type $(\alpha \rightarrow *) \rightarrow *$; quantifying over it is third-order. In λC , all of these are well-formed:

λC type	Logical reading	Order
$\prod x:\alpha. P\ x$	$\forall x. P(x)$	1st
$\prod P:(\alpha \rightarrow *). R(P)$	$\forall P. R(P)$	2nd
$\prod F:((\alpha \rightarrow *) \rightarrow *). \dots$	$\forall F. \dots$	3rd

There is no bound on the order. This is **higher-order intuitionistic predicate logic**.

The Curry–Howard table: propositions

λC	Logic
$A : *$	A is a proposition
$t : A$	t is a proof of A
$A \rightarrow B$	implication: if A then B
$\Pi x:\alpha. P(x)$	$\forall x \in \alpha. P(x)$
$\Pi P:(\alpha \rightarrow *). R(P)$	for all predicates P on α , $R(P)$

Each line is a Π -type (or its non-dependent special case $\rightarrow$). The rows differ in *what is being quantified over*: a term, a type, a predicate. λC allows all of them.

The Curry–Howard table: proofs

λC	Logic
$\lambda x:A. t$	proof of $A \rightarrow B$: assume A , derive B
$f a$	modus ponens: from $A \rightarrow B$ and A , deduce B
$\lambda x:\alpha. p$	proof of $\forall x. P(x)$: given any x , produce a proof of $P(x)$
$f a$	instantiation: from $\forall x. P(x)$ and a specific a , deduce $P(a)$

Every line was already present in some subsystem. The novelty of λC is that they coexist and can be freely combined: a single term can quantify over a type, then a term, then a predicate, nesting all three forms.

The logical hierarchy

System	Logic	Can quantify over...
$\lambda \rightarrow$ (STLC)	propositional	nothing
λP	1st-order predicate	terms
$\lambda 2$ (System F)	2nd-order propositional	types (= propositions)
λC	higher-order predicate	terms, types, predicates, ...

Each step in the course has enlarged the logic. λC is the culmination: it can express any statement of higher-order predicate logic, and a term of that type is a proof.

Let us now put this to work. We will define equality, encode the standard connectives of logic, and then ask whether we can derive the induction principle.

A taste: one type that uses everything

Before diving into the constructions, consider this type (in context $\alpha : *$):

$$\prod P : (\alpha \rightarrow *) . (\prod x : \alpha . P\ x) \rightarrow (\prod x : \alpha . P\ x)$$

Read it logically: *for every predicate P on α , if P holds of all elements, then P holds of all elements*. This is a tautology. What is the proof?

A taste: one type that uses everything

Before diving into the constructions, consider this type (in context $\alpha : *$):

$$\Pi P : (\alpha \rightarrow *). (\Pi x : \alpha. P\ x) \rightarrow (\Pi x : \alpha. P\ x)$$

Read it logically: *for every predicate P on α , if P holds of all elements, then P holds of all elements*. This is a tautology. What is the proof?

$$\lambda P : (\alpha \rightarrow *). \lambda f : (\Pi x : \alpha. P\ x). f$$

The proof is the identity: if you already have a proof that P holds everywhere, just return it. But the *type* of this identity requires both $(\square, *)$ (to quantify over P) and $(*, \square)$ (for $P\ x$, a type depending on the term x). Neither $\lambda 2$ nor λP alone can type this trivial term.

Section 3

Part III: Leibniz Equality

What is propositional equality?

In STLC and System F, there is no way to *state* that two terms are equal. The type $\text{Nat} \rightarrow \text{Nat}$ tells us that a function maps naturals to naturals, but there is no type expressing $f\ 3 = 5$ or $f = g$.

Propositional equality is a type $\text{Eq } \alpha\ x\ y$ that is inhabited exactly when x and y are equal. A proof of $\text{Eq } \alpha\ x\ y$ is a *term* — a witness that x and y are the same. This turns equality into something the type system can reason about: functions can require equality proofs as arguments, and theorems can produce them as outputs.

In λP , we could form types mentioning terms, but we had no polymorphism over predicates, so we could not *define* equality. In λC , we can.

The definition

Given $\alpha : *$ and $x, y : \alpha$:

$$\text{Eq } \alpha \ x \ y \equiv \prod P : (\alpha \rightarrow *). P \ x \rightarrow P \ y$$

This says: x equals y if, for every property P , whenever P holds of x , it also holds of y . In other words: x and y are **indiscernible** — no predicate can tell them apart. This is Leibniz's **identity of indiscernibles**, turned into a type.

Why this type needs λC

$$\text{Eq } \alpha \ x \ y = \Pi P:(\alpha \rightarrow *). P \ x \rightarrow P \ y$$

Three rules contribute:

- $(\square, *)$: the binder $\Pi P:(\alpha \rightarrow *)$ quantifies over predicates. The kind $\alpha \rightarrow *$ lives in $\square$; the codomain $P \ x \rightarrow P \ y$ lives in $*$.
- $(*, \square)$: the types $P \ x$ and $P \ y$ depend on terms x and y .
- $(*, *)$: the arrow $P \ x \rightarrow P \ y$ is an ordinary function type.

This type is unstatable in any proper subsystem of λC .

What does $\text{Eq } \alpha \ x \ x$ look like? Substituting $y = x$:

$$\text{Eq } \alpha \ x \ x = \prod P : (\alpha \rightarrow *). P \ x \rightarrow P \ x$$

This is the type of the identity function on $P \ x$: given a proof that P holds of x , return the same proof. So the proof of reflexivity is:

$$\text{refl} = \lambda \alpha : *. \lambda x : \alpha. \lambda P : (\alpha \rightarrow *). \lambda h : P \ x. h$$

The proof is trivial — just the identity. This is a good sign: it means the definition captures something natural. But the *type* of refl , namely $\prod \alpha : *. \prod x : \alpha. \text{Eq } \alpha \ x \ x$, is already a statement that only λC can express.

Symmetry: the goal

This definition of equality satisfies all the properties one expects: reflexivity, symmetry, transitivity, and substitutivity (if $x = y$ and $P x$, then $P y$ — which is the definition itself). Reflexivity is trivial. Transitivity is a one-liner (an exercise later). The interesting one is **symmetry**, because it requires a non-obvious idea.

We want:

$$\text{sym} : \Pi\alpha : * . \Pi x : \alpha . \Pi y : \alpha . \text{Eq } \alpha x y \rightarrow \text{Eq } \alpha y x$$

Unfolding Eq: given $e : \Pi P : (\alpha \rightarrow *) . P x \rightarrow P y$, produce a term of type $\Pi P : (\alpha \rightarrow *) . P y \rightarrow P x$.

Notice the difficulty: e converts evidence *about* x into evidence *about* y . We need to go the **other direction**.

Symmetry: what tools do we have?

We have $e : \prod P : (\alpha \rightarrow *) . P\ x \rightarrow P\ y$. The crucial feature: e works for **any** predicate P . We get to choose which predicate to feed it.

Our goal is to produce, for a given P , a function $P\ y \rightarrow P\ x$.

Symmetry: what tools do we have?

We have $e : \prod P : (\alpha \rightarrow *). P\ x \rightarrow P\ y$. The crucial feature: e works for **any** predicate P . We get to choose which predicate to feed it.

Our goal is to produce, for a given P , a function $P\ y \rightarrow P\ x$.

So the game is: find a predicate $Q : \alpha \rightarrow *$ such that:

- $Q\ x$ is something we can easily provide (we start from x 's side)
- $Q\ y$ is the thing we actually want, namely $P\ y \rightarrow P\ x$

Then $e\ Q : Q\ x \rightarrow Q\ y$ bridges the gap, as long as we can supply a term of type $Q\ x$.

Symmetry: the predicate trick

We need Q such that $Q\ y = P\ y \rightarrow P\ x$. A natural choice: define Q so that $Q\ z = P\ z \rightarrow P\ x$ for any z :

$$Q = \lambda z:\alpha. P\ z \rightarrow P\ x$$

Check the two requirements:

- $Q\ x = P\ x \rightarrow P\ x$. Can we inhabit this? **Yes**: the identity $\lambda h:P\ x. h$.
- $Q\ y = P\ y \rightarrow P\ x$. This is exactly what we need. ✓

Symmetry: applying the trick

We have $Q = \lambda z:\alpha. P\ z \rightarrow P\ x$, and $e : \Pi P. P\ x \rightarrow P\ y$ works for any predicate. Apply e to Q :

$$e\ Q : Q\ x \rightarrow Q\ y = (P\ x \rightarrow P\ x) \rightarrow (P\ y \rightarrow P\ x)$$

Feed it the identity $\lambda h. h : P\ x \rightarrow P\ x$:

$$e\ Q\ (\lambda h. h) : P\ y \rightarrow P\ x$$

This is what we wanted: a function from $P\ y$ to $P\ x$.

Symmetry: the full term

Putting it all together:

$$\begin{aligned} \text{sym} &= \lambda\alpha:*. \lambda x:\alpha. \lambda y:\alpha. \lambda e:\text{Eq } \alpha \ x \ y. \\ &\quad \lambda P:(\alpha \rightarrow *). \lambda h:P \ y. \\ &\quad \underbrace{e \ (\lambda z:\alpha. P \ z \rightarrow P \ x)}_{\text{instantiate } e \text{ at } Q} \ \underbrace{(\lambda h':P \ x. h')}_{\text{proof of } Q \ x} \ \underbrace{h}_{\text{apply to } h:P \ y} \end{aligned}$$

Three parts after the binders: instantiate e at Q , feed it the identity, apply the result to h .

Exercise: prove transitivity. Given $e_1 : \text{Eq } \alpha \ x \ y$ and $e_2 : \text{Eq } \alpha \ y \ z$, the proof is $\lambda P. \lambda h. e_2 \ P \ (e_1 \ P \ h)$. Why? (Hint: transitivity is composition — no predicate trick needed.)

Section 4

Part IV: Encoding Logic in λC

The programme

We have seen that λC can define equality. But equality is only one part of logic. The standard logical toolkit includes:

- existential quantification ($\exists x. P(x)$)
- conjunction ($A \wedge B$) and disjunction ($A \vee B$)
- falsity ($\perp$) and negation ($\neg A$)

In System F, we encoded the propositional connectives using $\Pi\alpha:*. \lambda C$ has dependent types on top of that. Can we now encode the *full* apparatus of predicate logic — including first-order existentials?

And the deeper question: System F let us *define* natural numbers via Church encodings and derive iteration. Can λC go further and derive **induction**?

Let us take the connectives first, then turn to induction.

The idea behind the encodings

Recall how the System F encoding of conjunction works:

$$A \wedge B \equiv \forall \gamma. (A \rightarrow B \rightarrow \gamma) \rightarrow \gamma$$

The idea: a proof of $A \wedge B$ is a black box that, given *any* target type γ and a function that can turn proofs of A and B into a γ , produces a γ . The only way it can do this is by secretly holding an $a : A$ and a $b : B$ and feeding them to the function.

The whole type $\forall \gamma. (A \rightarrow B \rightarrow \gamma) \rightarrow \gamma$ reads: *for any type γ you choose, give me a function $k : A \rightarrow B \rightarrow \gamma$, and I will produce a γ* . The function k is a **continuation**: it says what to do with an A and a B . The proof must work for *every* γ , so its only strategy is to hold an $a : A$ and a $b : B$ internally and call k a b .

The same idea extends to all connectives, including existentials.

Existential quantification

Given $\alpha : *$ and $P : \alpha \rightarrow *$, the existential *there exists $x : \alpha$ such that $P(x)$* is:

$$\exists x:\alpha. P\ x \equiv \Pi\beta:*. (\Pi x:\alpha. P\ x \rightarrow \beta) \rightarrow \beta$$

This uses the same continuation-passing idea as the System F encodings. A proof of $\exists x. P\ x$ is a function that says: *tell me what you want to conclude (β), and show me how to get there from any witness ($\Pi x:\alpha. P\ x \rightarrow \beta$), and I will produce your conclusion.* The only way to produce β from the given function is to feed it a specific witness and proof — so the existential proof must contain one internally.

Existential: introduction

To prove $\exists x:\alpha. P\ x$, we supply a specific witness $a : \alpha$ and a proof $p : P\ a$:

$$\text{pack } a\ p = \lambda\beta:*. \lambda k:(\Pi x:\alpha. P\ x \rightarrow \beta). k\ a\ p$$

Check the types: $k : \Pi x:\alpha. P\ x \rightarrow \beta$, so $k\ a : P\ a \rightarrow \beta$ (substituting $x := a$), so $k\ a\ p : \beta$. ✓

Existential: elimination

In logic, the $\exists$ -elimination rule says: if you know $\exists x. P(x)$, and you can derive a conclusion β from $P(x)$ **for an arbitrary** x , then you may conclude β . You do not know which witness it is, so your argument must work for any of them.

This is exactly what the encoding gives. Given $e : \exists x:\alpha. P\ x$, you choose your desired conclusion β and provide a function $k : \Pi x:\alpha. P\ x \rightarrow \beta$ that derives β from $P(x)$ for any x . Then $e\ \beta\ k : \beta$.

Dependent pairs (Σ -types)

The existential encoding is a special case of a more general construction. Given $A : *$ and $B : A \rightarrow *$:

$$\Sigma x:A. B(x) \equiv \Pi \gamma : *. (\Pi x:A. B(x) \rightarrow \gamma) \rightarrow \gamma$$

A value of this type is a **dependent pair**: it packages an $a : A$ together with a $b : B(a)$, where the type of the second component depends on the value of the first.

Under Curry–Howard, $\Sigma x:\alpha. P x$ is the existential $\exists x. P(x)$: the pair contains a witness and a proof. As data, $\Sigma n:\text{Nat}. \text{Even } n$ is a natural number bundled with evidence that it is even.

When B does not depend on x , this reduces to an ordinary pair $A \times B$. The Σ -type is the dependent generalization of the pair type, just as Π is the dependent generalization of $\rightarrow$.

All connectives from Π alone

$$\perp \equiv \Pi\alpha:*. \alpha$$

$$\neg A \equiv A \rightarrow \perp$$

$$A \wedge B \equiv \Pi\gamma:*. (A \rightarrow B \rightarrow \gamma) \rightarrow \gamma$$

$$A \vee B \equiv \Pi\gamma:*. (A \rightarrow \gamma) \rightarrow (B \rightarrow \gamma) \rightarrow \gamma$$

$$\exists x:\alpha. P\ x \equiv \Pi\gamma:*. (\Pi x:\alpha. P\ x \rightarrow \gamma) \rightarrow \gamma$$

The propositional connectives ($\perp$, $\neg$, $\wedge$, $\vee$) are the System F encodings, reused verbatim — they require only $(\square, *)$. The existential is new: the inner function $\Pi x:\alpha. P\ x \rightarrow \gamma$ depends on a term, which requires $(*, \square)$.

Every connective follows the same pattern: to *prove* it, package your data into a function; to *reason from* it, supply the function with the appropriate arguments.

Conjunction and disjunction: explicit terms

$A \wedge B$: ; pair $a\ b = \lambda\gamma. \lambda k. k\ a\ b$.

Projections: $\text{fst } p = p\ A\ (\lambda a. \lambda b. a)$ and $\text{snd } p = p\ B\ (\lambda a. \lambda b. b)$.

$A \vee B$: ; inl $a = \lambda\gamma. \lambda f. \lambda g. f\ a$. ; inr $b = \lambda\gamma. \lambda f. \lambda g. g\ b$.

Case split: $d\ C\ f\ g$.

Verify: $\text{fst } (\text{pair } a\ b) \rightarrow_{\beta} (\lambda a. \lambda b. a)\ a\ b \rightarrow_{\beta} a$. ✓

This is the entire logical repertoire. λC needs no primitive connectives: every logical operation is a particular use of Π , λ , and application. That answers the first question from our programme: λC encodes the full apparatus of predicate logic. Now for the deeper question: can it derive **induction**?

Section 5

Part V: The Induction Principle

The question

Church numerals from System F still work in λC :

$$\text{Nat}_{\text{Ch}} = \Pi\alpha: * . (\alpha \rightarrow \alpha) \rightarrow \alpha \rightarrow \alpha$$

And now λC can *state* the induction principle (impossible in System F, since $(*, \square)$ was absent):

$$\text{Ind} = \Pi P: (\text{Nat}_{\text{Ch}} \rightarrow *) . P \bar{0} \rightarrow (\Pi m . P m \rightarrow P (S m)) \rightarrow \Pi n: \text{Nat}_{\text{Ch}} . P n$$

Read it logically: for any property P of natural numbers, if P holds at zero and is preserved by successor, then P holds everywhere. Can we construct a term of this type?

What a Church numeral can do

A Church numeral $n : \prod \alpha : * . (\alpha \rightarrow \alpha) \rightarrow \alpha \rightarrow \alpha$ takes three arguments:

- a type α
- a step function $f : \alpha \rightarrow \alpha$
- a starting point $x : \alpha$

and produces $f^n(x) : \alpha$ — the result of applying f to x exactly n times.

The crucial point: the type α is **fixed throughout the iteration**. Every intermediate value has the same type α . The step function f always goes from α to α .

What induction requires

Induction is different. To prove $P\ n$ by induction, we need:

- a base case $p_0 : P\ \bar{0}$
- a step $s : \Pi m. P\ m \rightarrow P\ (S\ m)$

Now look at what happens at each step:

Step	Input type	Output type
0	(start)	$P\ \bar{0}$
1	$P\ \bar{0}$	$P\ \bar{1}$
2	$P\ \bar{1}$	$P\ \bar{2}$
k	$P\ \overline{k-1}$	$P\ \bar{k}$

The type **changes at every step**. The step function does not have type $\alpha \rightarrow \alpha$ for any fixed α — it has type $P\ m \rightarrow P\ (S\ m)$, which is a *different type* for each m .

The mismatch

This is the core problem:

Iteration (what Church numerals do) $f : \alpha \rightarrow \alpha$

same type in, same type out

Induction (what we need) $s : P\ m \rightarrow P\ (S\ m)$

type changes at every step

A Church numeral iterates a function $f : \alpha \rightarrow \alpha$ for a **single fixed type** α . Induction requires a step function whose type **depends on the current position**. These are fundamentally different operations. No choice of α can bridge the gap, because the Church numeral's type *commits* to a single α before the iteration begins.

What we already have

We have had Nat as a primitive type with constructors since System T:

$$0 : \text{Nat} \quad S : \text{Nat} \rightarrow \text{Nat}$$

And System T came with a recursor R . But its type was non-dependent:

$$R : \alpha \rightarrow (\text{Nat} \rightarrow \alpha \rightarrow \alpha) \rightarrow \text{Nat} \rightarrow \alpha$$

The same fixed α throughout — that gives iteration, not induction.

What we can state but not inhabit

In λC , we can *state* the induction principle as a type:

$$\prod P : (\text{Nat} \rightarrow *) . P\ 0 \rightarrow (\prod m : \text{Nat} . P\ m \rightarrow P\ (S\ m)) \rightarrow \prod n : \text{Nat} . P\ n$$

This is a well-formed type in λC : it requires $(\square, *)$ for $\prod P$ and $(*, \square)$ for $P\ n$. System T could not even write this type.

But we cannot **inhabit** it: as we just saw, no λ -term has this type when Nat is Church-encoded. We can state the induction principle but not prove it.

What we add: a primitive recursor

The fix is to add, for each inductively defined type, a **primitive recursor** `rec` whose type is exactly the induction principle:

$$\text{rec} : \Pi P : (\text{Nat} \rightarrow *). P\ 0 \rightarrow (\Pi m : \text{Nat}. P\ m \rightarrow P\ (S\ m)) \rightarrow \Pi n : \text{Nat}. P\ n$$

This is a new **constant**, added to the system. It is not defined as a λ -term — it is postulated, together with **computation rules** that tell the type checker how it reduces. The recursor is the essential novelty; the constructors alone were not enough.

The computation rules

The recursor reduces by **pattern matching on the last argument**:

$$\begin{aligned}\text{rec } P \, p_0 \, s \, 0 &\equiv p_0 \\ \text{rec } P \, p_0 \, s \, (\text{S } n) &\equiv s \, n \, (\text{rec } P \, p_0 \, s \, n)\end{aligned}$$

These are new reduction rules, added alongside β . When the type checker needs to compare types, it applies these rules just as it applies β -reduction.

The first rule says: at zero, return the base case. The second says: at a successor, apply the step function to the predecessor and the recursive result. This is ordinary recursion, but with a *dependent* type.

Why constructors matter

Why can the recursor's rules fire when n is a variable? Because they match on **constructors**.

Every term of type `Nat` is built from `0` and `S`. When the recursor sees `S  $n$` , the constructor `S` is visible on the outside, so the second rule fires — even though n is unknown. The recursor does not need to know what n is; it only needs to see the outermost `S`.

A Church numeral is a *function*, not a constructor application. There is nothing on the outside to match against. This is why the recursor, with rules that match on constructors, solves the problem that Church encodings could not.

How the types work out

The recursor's type says: $\text{rec } P \, p_0 \, s : \prod n:\text{Nat}. P \, n$. So:

$$\text{rec } P \, p_0 \, s \, (\text{S } n) : P \, (\text{S } n)$$

The computation rule reduces this to $s \, n \, (\text{rec } P \, p_0 \, s \, n)$. Does this have the right type?

Checking the reduced term

In the expression $s\ n\ (\text{rec } P\ p_0\ s\ n)$:

- $\text{rec } P\ p_0\ s\ n : P\ n$ (the recursive call)
- $s\ n : P\ n \rightarrow P\ (S\ n)$ (the step, instantiated at n)
- $s\ n\ (\text{rec } P\ p_0\ s\ n) : P\ (S\ n)$ ✓ (function applied to argument)

The recursive call has type $P\ n$ and the result has type $P\ (S\ n)$ — the type *changes* at each step, tracked by the dependent type of s . This is exactly what iteration with a fixed α could not achieve.

From λC to CIC

The combination of λC with primitive inductive types and their dependent recursors is the **Calculus of Inductive Constructions** (CIC). This is the core of Lean and Coq.

The pattern generalizes beyond Nat: for *any* inductively defined type (lists, vectors, trees, equality, ...), we get constructors and a dependent recursor. The recursor's type is always the *induction principle* for that type, and its computation rules always match on constructors.

λC provides the logical framework: dependent types, polymorphism, higher-order quantification. The recursors provide the computational power: pattern matching, recursion, and the definitional equalities that make type checking work. This is the subject of Lecture 18.

Section 6

Summary

What we covered

- 1 **λC defined:** $\mathcal{R} = \{(*, *), (\Box, *), (\Box, \Box), (*, \Box)\}$ — all four rules, same syntax and inference rules.
- 2 **Historical motivation:** Coquand and Huet's goal of a single system for defining data, writing programs, stating properties, and proving them.
- 3 **Curry–Howard:** λC corresponds to higher-order intuitionistic predicate logic — quantification over terms, types, predicates, and beyond, all as Π -types.
- 4 **Leibniz equality:** defined as $\Pi P: (\alpha \rightarrow *) . P\ x \rightarrow P\ y$; reflexivity, symmetry (the predicate trick), transitivity (exercise).
- 5 **Encoding logic:** existentials, Σ -types, and all propositional connectives from Π alone.
- 6 **The induction gap:** Church encodings support iteration but not dependent elimination — motivating inductive types.

Next lecture: the Calculus of Constructions, continued.

We study the system's metatheory: strong normalization and its consequences (consistency, decidable type checking, termination), impredicativity (controlled self-reference), and the computational strength hierarchy (what λC can and cannot prove about itself). We also introduce **pure type systems**, which generalize the cube beyond its eight vertices.